Tractable Problems in AI Security via Formal Methods

Forall R&D:

Guiding AI safety orgs through the formal methods explosion.

What formal methods can and can't claim

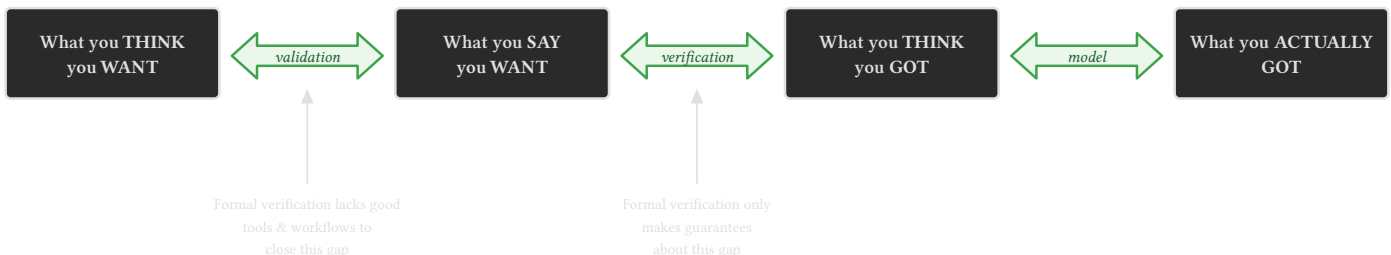

Figure 1: After Evan Miyazono. FM closes the middle gap only.

- **Elicitation gap** (left): is the spec the thing we actually wanted? Out of scope for the proof.
- **Verification gap** (middle): does the implementation meet the spec? This is what FM does.
- **Modeling gap** (right): does the proof's model of the hardware/OS/network match reality?

The ML training and inference stack

The tractable problems

17 entries on the website. Two flavors:

The tractable problems

17 entries on the website. Two flavors:

- **Enablers** unbottleneck a class of downstream work. Examples: adversarial robustness of FM tools, neuralese governance, spec elicitation.
- **Widgets** are concrete, scoped deliverables. Examples: seL4 GPU drivers, OCI runtime hardening, verified network tap, weight integrity.

The tractable problems

17 entries on the website. Two flavors:

- **Enablers** unbottleneck a class of downstream work. Examples: adversarial robustness of FM tools, neuralese governance, spec elicitation.
- **Widgets** are concrete, scoped deliverables. Examples: seL4 GPU drivers, OCI runtime hardening, verified network tap, weight integrity.

An enabler with many dependents should be addressed early. A widget contingent on an unsolved enabler should be sequenced accordingly.

Deep dive: five widgets

I'll spend a slide or two on each.

1. Verified network tap
2. OCI runtime hardening
3. seL4 native GPU drivers
4. AI control via proof-carrying code
5. Adversarial robustness of FM

Widget 1: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

Widget 1: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

The artifact: a passive (or active OEO) tap on the fiber, with an FPGA capture path proven against an LTL contract. North-south at the datacenter edge, east-west for storage, sampled for compute.

Widget 1: Verified network tap

The threat: the SDN controller and fabric manager on a GPU cluster *are* the thing whose compromise we are worried about. Their own logs cannot be evidence.

The artifact: a passive (or active OEO) tap on the fiber, with an FPGA capture path proven against an LTL contract. North-south at the datacenter edge, east-west for storage, sampled for compute.

Costs (Cankaya et al., 2026): under 0.01% of facility upfront for the edge tap; 0.3–0.75% for east-west. FPGA capture at 400 Gbps is a solved hardware problem (Arista 7130, Deutsche Börse).

Verified network tap: the LTL contract

$$\square(\text{live\_in}(p) \rightarrow \Diamond_{\leq D}\ \text{live\_out}(p)) \qquad \text{(P1) forwarding completeness}$$

$$\square(\text{live\_out}(p) \rightarrow \blacklozenge\ \text{live\_in}(p)) \qquad \text{(P2) forwarding fidelity}$$

$$\square(\text{live\_in}(p) \rightarrow \Diamond_{\leq D'}\ \text{mon}(\text{image}(p))) \qquad \text{(P3) mirror completeness}$$

$$\square(\text{mon}(r) \rightarrow \blacklozenge(\exists p.\ \text{live\_in}(p) \wedge \text{image}(p) = r)) \qquad \text{(P4) mirror soundness}$$

$$\square(\text{live\_out}(p) \rightarrow \neg\ \text{depends-on}(p, \text{mon-input})) \qquad \text{(P5) no back-channel}$$

$$\square(\text{fail} \rightarrow \square(\text{live\_in}(p) \rightarrow \Diamond_{\leq D}\ \text{live\_out}(p))) \qquad \text{(P6) fail-to-wire}$$

Verified network tap: the LTL contract

$$\square(\text{live\_in}(p) \to \Diamond_{\leq D}\ \text{live\_out}(p)) \qquad \text{(P1) forwarding completeness}$$

$$\square(\text{live\_out}(p) \to \blacklozenge\ \text{live\_in}(p)) \qquad \text{(P2) forwarding fidelity}$$

$$\square(\text{live\_in}(p) \to \Diamond_{\leq D'}\ \text{mon}(\text{image}(p))) \qquad \text{(P3) mirror completeness}$$

$$\square(\text{mon}(r) \to \blacklozenge(\exists p.\ \text{live\_in}(p) \wedge \text{image}(p) = r)) \qquad \text{(P4) mirror soundness}$$

$$\square(\text{live\_out}(p) \to \neg\ \text{depends-on}(p, \text{mon-input})) \qquad \text{(P5) no back-channel}$$

$$\square(\text{fail} \to \square(\text{live\_in}(p) \to \Diamond_{\leq D}\ \text{live\_out}(p))) \qquad \text{(P6) fail-to-wire}$$

P5 is the noninterference property that defeats verifier collusion. P6 is fail-open: a fault must downgrade to a transparent pass-through, never a link cut.

Widget 2: OCI runtime hardening

`runc` was never designed to resist an adversary *inside* the container. The CVE record reflects this.

Widget 2: OCI runtime hardening

`runc` was never designed to resist an adversary *inside* the container. The CVE record reflects this.

When the workload is an agent with tool use

Widget 2: OCI runtime hardening

writes code, executes it

Widget 2: OCI runtime hardening

“process isolation” is not a security boundary.

Widget 2: OCI runtime hardening

“process isolation” is not a security boundary.

Empirical work: BoxArena (Dougherty 2026) inverts the offensive framing. Fix the attacker model, vary the runtime. Five attack surfaces (fs, socket, process, network, syscall) across `runc`, `runsc`, `crun`, `Kata`.

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

- A formal model of the Linux ABI surface a hardened container actually uses (~80 syscalls, not 350+).
- A specification of the confinement policy (drop caps, read-only rootfs, seccomp allowlist, no-new-privs).
- A proof that the runtime's state machine preserves the policy across all reachable states.

OCI hardening: what the verified version looks like

Decomposes into tractable pieces:

- A formal model of the Linux ABI surface a hardened container actually uses (~80 syscalls, not 350+).
- A specification of the confinement policy (drop caps, read-only rootfs, seccomp allowlist, no-new-privs).
- A proof that the runtime's state machine preserves the policy across all reachable states.

Lineage: `CLInc` (Bevier 1989, gates to apps), `seL4` (Klein 2009, microkernel functional correctness). Same shape, different target.

Widget 3: seL4 native GPU drivers

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Widget 3: seL4 native GPU drivers

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Candidates to host an ML-grade GPU stack:

- `seL4` — most complete proof, no GPU support at all.
- `NOVA` — partial proof covers concurrency and weak memory, host/guest split built in.

Widget 3: seL4 native GPU drivers

Multi-tenant GPU workloads run on hypervisors whose TCB is orders of magnitude too large to verify, with a proprietary ring-0 driver from the vendor.

Candidates to host an ML-grade GPU stack:

- `seL4` — most complete proof, no GPU support at all.
- `NOVA` — partial proof covers concurrency and weak memory, host/guest split built in.

The methodology (separation logic, abstract HW model) is settled. The blocker is that **the abstract hardware model does not exist**. No GPU vendor publishes a machine-checkable spec of their command processor.

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (`NVK`/ `Nouveau` or an `AMDGPU` subset).

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (`NVK`/ `Nouveau` or an `AMDGPU` subset).

Property: *no sequence of guest-supplied command packets causes the driver to program an IOMMU mapping or issue a DMA outside the guest's declared memory region.*

seL4 GPU: the actual scope

Pick the smallest useful surface: command submission for one open stack (`NVK`/ `Nouveau` or an `AMDGPU` subset).

Property: *no sequence of guest-supplied command packets causes the driver to program an IOMMU mapping or issue a DMA outside the guest's declared memory region.*

Two stopping points:

- Verified module against a stubbed hardware model — shippable as reference.
- Same driver against a model co-developed with the vendor or with REMS (Sewell) — the research contribution.

Widget 4: AI control via proof-carrying code

Two literatures, one problem. Keep an untrusted producer behind an interface; let a small trusted checker decide what passes.

Widget 4: AI control via proof-carrying code

Two literatures, one problem. Keep an untrusted producer behind an interface; let a small trusted checker decide what passes.

- **PCC** (Necula 1997): every action arrives with a machine-checkable certificate. Trusted base = proof kernel.
- **AI Control** (Greenblatt 2024, Bhatt 2025): trusted monitor, scarce audit budget, safety–usefulness Pareto frontier.

Widget 4: AI control via proof-carrying code

Two literatures, one problem. Keep an untrusted producer behind an interface; let a small trusted checker decide what passes.

- **PCC** (Necula 1997): every action arrives with a machine-checkable certificate. Trusted base = proof kernel.
- **AI Control** (Greenblatt 2024, Bhatt 2025): trusted monitor, scarce audit budget, safety–usefulness Pareto frontier.

Midpoint: untrusted agent emits code *and a proof*. The boundary rejects anything that does not typecheck. Trusted base = kernel + human-written spec.

AI Control via PCC: the key unknown

The **verification burden** k: ratio of proof-synthesis cost to program-synthesis cost.

AI Control via PCC: the key unknown

The **verification burden** k: ratio of proof-synthesis cost to program-synthesis cost.

- Small k everywhere $\longrightarrow$ AI-control-via-PCC is the right architecture for most widgets in this document.
- Large k, or small only for spec-shapes nobody writes $\longrightarrow$ widgets must be verified individually.

AI Control via PCC: the key unknown

The **verification burden** k: ratio of proof-synthesis cost to program-synthesis cost.

- Small k everywhere $\longrightarrow$ AI-control-via-PCC is the right architecture for most widgets in this document.
- Large k, or small only for spec-shapes nobody writes $\longrightarrow$ widgets must be verified individually.

Three subproblems determine where k lands:

1. Proofs as a matter of degree (graded gatekeepers, not pass/fail).
2. Specialist blue-team models for narrow Hoare fragments.
3. Who pays for tooling — red team's compliance budget, or blue team's?

Widget 5 (enabler): Adversarial robustness of FM

`Agda`'s GitHub issue label "false" tracks proofs of false the type checker has admitted. 10 open, 76 closed at last count.

Widget 5 (enabler): Adversarial robustness of FM

`Agda`'s GitHub issue label "false" tracks proofs of false the type checker has admitted. 10 open, 76 closed at last count.

If the ITP's soundness can be broken by a clever agent, every verified artifact built on top inherits the vulnerability.

Widget 5 (enabler): Adversarial robustness of FM

`Agda`'s GitHub issue label "false" tracks proofs of false the type checker has admitted. 10 open, 76 closed at last count.

If the ITP's soundness can be broken by a clever agent, every verified artifact built on top inherits the vulnerability.

This is the enabler with the most dependents. A scheming model that finds a kernel zero-day defeats the entire stack of widgets in one shot. It also gates AI-control-via-PCC

Widget 5 (enabler): Adversarial robustness of FM

the trusted base in that protocol *is* the kernel.

Adversarial robustness: the sketch

Soundness bugs are downstream of the complexity of dependent type theory. The proof checker (kernel) is large.

Adversarial robustness: the sketch

Soundness bugs are downstream of the complexity of dependent type theory. The proof checker (kernel) is large.

One direction: express the language of the ITP in a smaller, simpler theory. Model a complicated dependently-typed kernel inside `Isabelle` or `ACL2`.

Adversarial robustness: the sketch

Soundness bugs are downstream of the complexity of dependent type theory. The proof checker (kernel) is large.

One direction: express the language of the ITP in a smaller, simpler theory. Model a complicated dependently-typed kernel inside `Isabelle` or `ACL2`.

`Lean` Arena (arena.lean-lang.org) is the live red-team venue.

Adversarial robustness: the sketch

Soundness bugs are downstream of the complexity of dependent type theory. The proof checker (kernel) is large.

One direction: express the language of the ITP in a smaller, simpler theory. Model a complicated dependently-typed kernel inside `Isabelle` or `ACL2`.

`Lean` Arena (arena.lean-lang.org) is the live red-team venue.

Also a governance question: who is the maintainer when the bug is found by an agent at 3am?

Why this is hard to fund

The honest problem: FM competes in a market where the unverified alternative is *free*.

Why this is hard to fund

The honest problem: FM competes in a market where the unverified alternative is *free*.

A startup choosing between a verified hypervisor and `KVM`, or a verified container runtime and `runc`, is comparing a paid product against a zero-marginal-cost incumbent that is good enough for the threat model they think they have.

Why this is hard to fund

The honest problem: FM competes in a market where the unverified alternative is *free*.

A startup choosing between a verified hypervisor and `KVM`, or a verified container runtime and `runc`, is comparing a paid product against a zero-marginal-cost incumbent that is good enough for the threat model they think they have.

FM talent today is concentrated in avionics, defense, automotive

Why this is hard to fund

domains where regulators force the comparison to be against a counterfactual incident, not against the free alternative.

Why AI infra inverts the calculation

AI infra is one of the few private-sector settings where:

Why AI infra inverts the calculation

AI infra is one of the few private-sector settings where:

- The counterfactual incident is *large*: weight exfiltration, training-data poisoning, agentic container escape. Losses on the order of training cost.
- The customers (frontier labs, regulated deployers) have both budget and risk model.

Why AI infra inverts the calculation

AI infra is one of the few private-sector settings where:

- The counterfactual incident is *large*: weight exfiltration, training-data poisoning, agentic container escape. Losses on the order of training cost.
- The customers (frontier labs, regulated deployers) have both budget and risk model.

Making the business case requires *writing it down*. Per-widget: name the threat closed, the alternative being compared against, the lift to ship.

Why AI infra inverts the calculation

AI infra is one of the few private-sector settings where:

- The counterfactual incident is *large*: weight exfiltration, training-data poisoning, agentic container escape. Losses on the order of training cost.
- The customers (frontier labs, regulated deployers) have both budget and risk model.

Making the business case requires *writing it down*. Per-widget: name the threat closed, the alternative being compared against, the lift to ship.

That is what the document is.

What to do this weekend

If you came here to build, the hackathon-shaped subset:

What to do this weekend

If you came here to build, the hackathon-shaped subset:

- **Reproduce a BoxArena entry.** Pick a runtime, write the harness, run the five surfaces, publish the leaderboard row.
- **Spec a tiny piece of the OCI ABI in Lean or Rocq.** Even ten syscalls is useful.
- **Toy verified network tap in `Cryptol` or `SymbiYosys`.** P1, P2, P6 only. Stub the SerDes.
- **Measure k on one spec shape.** Pick context-window allocation. Pick one model. Publish the distribution.
- **Find a soundness bug in Agda or Lean.** Submit it to the Arena.

What to do over the next year

What to do over the next year

- If you're a **PL/FM researcher** looking for an AI-safety entry point: control protocols as a process calculus, or the verified OCI runtime.
- If you're an **ML person** worried about agentic deployment: BoxArena, sampler verification, context-window integrity.
- If you're **funding-shaped**: pay for the spec-elicitation work. It is the bottleneck the document points at.

What to do over the next year

- If you're a **PL/FM researcher** looking for an AI-safety entry point: control protocols as a process calculus, or the verified OCI runtime.
- If you're an **ML person** worried about agentic deployment: BoxArena, sampler verification, context-window integrity.
- If you're **funding-shaped**: pay for the spec-elicitation work. It is the bottleneck the document points at.

The website is the live menu: tractable.for-all.dev. The tags are layer × adversary × category. Filter to your shape.

Summary

Summary

- AI safety has an FM-shaped half nobody is staffing.
- The widgets are scoped. The principles are stable. The funding case inverts in this domain.
- 17 entries. Pick one.

Summary

- AI safety has an FM-shaped half nobody is staffing.
- The widgets are scoped. The principles are stable. The funding case inverts in this domain.
- 17 entries. Pick one.

Questions?

tractable.for-all.dev
quinn@for-all.dev

Backup: the full problem list

Enablers: adversarial robustness of FM, spec elicitation, neuralese governance, AI control via PCC.

Widgets, by layer:

- L1 (harness): verified input parsers, sampler verification, context-window integrity, capability accumulation, agent weird machines, loop termination, audit log integrity.
- L2 (framework): weight integrity (also L1/L4).
- L3 (orchestration): OCI runtime hardening, scheduler co-tenancy, edge policy verification, verified network tap.
- L4 (firmware): seL4 native GPU drivers.
- L5 (hardware): verified network tap (also L3).

Backup: the FM toolchain

What gets used in the widgets above:

Backup: the FM toolchain

What gets used in the widgets above:

- **ITPs:** `Lean`, `Rocq`, `Isabelle/HOL`, `Agda`, `ACL2`.
- **Separation logic:** `Iris` and descendants.
- **HW verification:** `Cryptol`, `Kami`, `SymbiYosys`.
- **Model checking:** `TLA+`, `Apalache`, `Spin`.
- **Translation validation:** for kernel binaries against contracts.

Backup: the FM toolchain

What gets used in the widgets above:

- **ITPs:** `Lean`, `Rocq`, `Isabelle/HOL`, `Agda`, `ACL2`.
- **Separation logic:** `Iris` and descendants.
- **HW verification:** `Cryptol`, `Kami`, `SymbiYosys`.
- **Model checking:** `TLA+`, `Apalache`, `Spin`.
- **Translation validation:** for kernel binaries against contracts.

The toolchain is not the bottleneck. The targets are.

Backup: what's out of scope

Backup: what's out of scope

- Proving the model itself. Different document.
- Scheduler optimization. Policy, not mechanism.
- Multimode fiber taps. Link budget does not admit passive splitting at high baud rates.
- Full `POSIX` proofs. API surface too large.
- Anything where the spec cannot factor cleanly from the implementation.